



MARMARA UNIVERSITY

FACULTY OF ENGINEERING

EE4065

Introduction to Embedded Image Processing

HOMEWORK 3

Submitted to: Prof. Dr. CEM ÜNSALAN

Due Date: 19.12.2025

	Dept.	STUDENT ID	NAME/SURNAME
1	EE	150720049	RUMEYSA ŞENOL
2	EE	150720996	FATİME EMİN

1.INTRODUCTION

This homework investigates fundamental embedded image processing operations on the STM32F446RE (Nucleo) platform. A grayscale image was embedded in the program memory and processed on the microcontroller using intensity-based segmentation (Otsu thresholding) and basic morphological operations. Since the microcontroller does not include a display, the processed image buffers were transmitted to a PC via UART and reconstructed in Python for visualization and evaluation.

2.METHODOLOGY

First, the input grayscale image was stored as a 1D uint8_t array (`image[]`) together with its dimensions in a header file. On the STM32, an Otsu-based threshold value was computed from the image histogram to automatically separate foreground and background. The image resolution was fixed to 64×64 pixels to fit the memory constraints of the embedded system and to allow real-time processing on the STM32F446RE. Using this threshold, a binary image (`thr[]`) was created. Then, morphological operations (dilation, erosion, opening, and closing) were applied using a 3×3 structuring element to observe how binary regions expand, shrink, and change connectivity. Finally, all resulting image arrays were transmitted through USART2 to the PC in raw byte form. A Python script received the data, reshaped it into 2D images, and displayed/saved the outputs for comparison.

3. PROCESS AND RESULTS

The input image was converted to grayscale and resized to 64×64 pixels. It was stored in the program memory as a one-dimensional unsigned 8-bit array:

$$image[i] \in [0,255], \quad i = 0, 1, 2, \dots (64 \times 64 - 1)$$

The representation is memory efficient and suitable for embedded processing. Original image is as shown below.



Question 1 - Otsu Thresholding

Otsu's method determines an optimal global threshold by maximizing the between-class **variance** of pixel intensities.

Let:

- be the number of gray levels,
- $p(i)$ be the normalized histogram,
- $\omega_0(t)$, $\omega_1(t)$ be class probabilities,
- $\mu_0(t)$, $\mu_1(t)$ be class means.

The between-class variance is defined as:

$$\sigma_b^2(t) = \omega_0(t)\omega_1(t)[\mu_0(t) - \mu_1(t)]^2$$

The threshold T is chosen such that:

$$T = \operatorname{argtmax} \sigma_b^2(t)$$

Computed Result

For the embedded image of size 64×64 , the STM32 computed:

$$T = 115$$

This value was obtained entirely on the microcontroller without any PC-side processing. Using the computed threshold, the binary image was generated as:

$$\text{thr}(x, y) = \begin{cases} 0, & \text{if image}(x, y) > T \\ 255, & \text{otherwise} \end{cases}$$

This operation separates the image into two intensity classes. The result is a binary image where object and background regions are clearly distinguishable.



Question 2 – Otsu’s method on Color image

In the second question, the color (RGB) image was resized to a fixed resolution and embedded into the STM32F446RE program memory in order to meet the limitations of the embedded system. Instead of converting the image to grayscale, the Red, Green, and Blue channels were processed independently to preserve channel-specific intensity information. Otsu’s thresholding algorithm was applied separately to each channel, resulting in three distinct threshold values corresponding to the R, G, and B components. Each channel was then binarized using its own threshold, and the resulting binary channel outputs were combined to form a color-segmented image. All computations were performed directly on the microcontroller, and the processed channel data were transmitted to a PC via UART for reconstruction and visualization, enabling verification of the embedded processing results without the use of a display unit.

Question 3 - Morphological Operations

All morphological operations were implemented using a 3×3 structuring element.

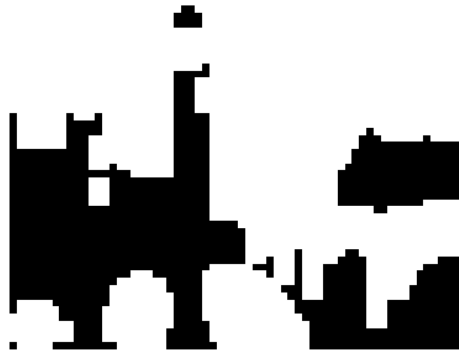
Dilation

Dilation expands the foreground region by adding pixels around object boundaries

$$A \oplus B = \{z \mid (B)z \cap A \neq \emptyset\}$$

This operation fills small gaps and connects nearby components.

Dilation



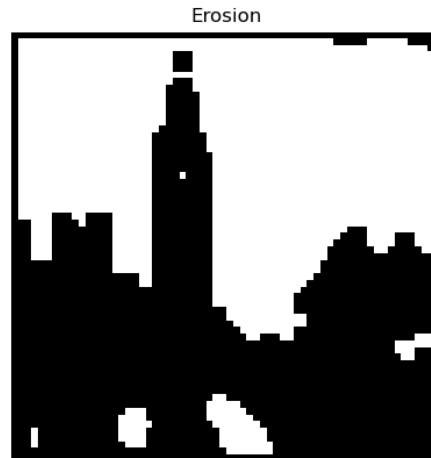
Dilation expands the foreground region based on the structuring element. As a result, boundaries become thicker and small gaps can be reduced. This operation tends to connect nearby components and fill narrow breaks, but it may also enlarge noise if isolated pixels exist.

Erosion

Erosion removes boundary pixels and shrinks foreground regions:

$$A \ominus B = \{z \mid (B)z \subseteq A\}$$

This helps remove small foreground noise but may break thin structures.



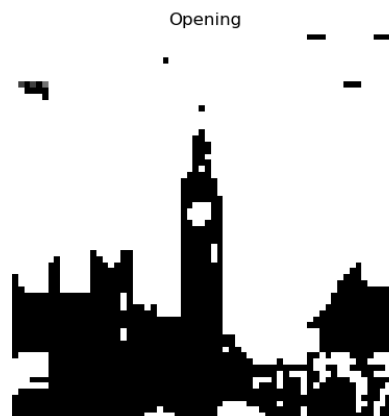
Erosion shrinks the foreground region by removing boundary pixels. In the output, object regions become thinner and small foreground artifacts may disappear. However, erosion can also break weakly connected structures and enlarge background holes within objects.

Opening (Erosion → Dilation)

Opening is defined as erosion followed by dilation:

$$A \circ B = (A \ominus B) \oplus B$$

It removes small isolated noise while preserving the main object shape.



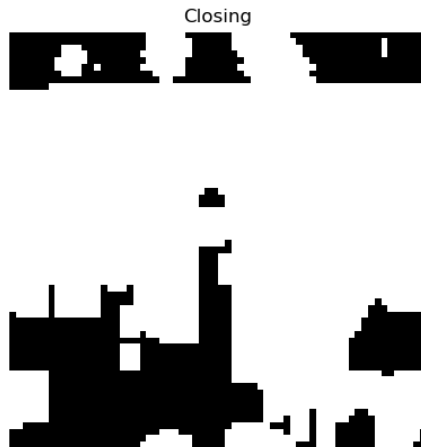
Opening removes small foreground noise while preserving the overall shape of larger objects. In our results, small isolated pixels were reduced compared to the thresholded image, and thin protrusions were smoothed, producing a cleaner binary representation.

Closing (Dilation → Erosion)

Closing is defined as dilation followed by erosion:

$$A \cdot B = (A \oplus B) \ominus B$$

It fills small holes inside objects and improves region connectivity.



Embedded Data Transmission

Since the STM32 does not support image display, the processed buffers were transmitted via **USART2** using the following protocol:

Order	Data type	Description
1	uint16	Image width (64)
2	uint16	Image height (64)
3	uint8	Otsu threshold (115)
4	uint8[]	Thresholded image
5	uint8[]	Dilation result
6	uint8[]	Erosion result
7	uint8[]	Opening result
8	uint8[]	Closing result

On the PC side, Python reconstructed the images and displayed them for analysis.

4.CONCLUSION

In this homework, fundamental image segmentation and morphological processing techniques were successfully implemented on an embedded STM32F446RE platform. An Otsu threshold value of 115 was automatically computed for a 64×64 grayscale image, enabling fully autonomous segmentation without manual tuning. Morphological operations demonstrated their expected effects on object boundaries, noise removal, and region connectivity. The results were verified by transmitting raw image buffers to a PC via UART and reconstructing them in Python. This study highlights the feasibility and effectiveness of performing essential image processing tasks directly on embedded hardware with limited resources.

The channel-wise processing approach used in Question 2 illustrates how pixel-level mathematical operations can be generalized to multi-channel images while remaining efficient for embedded systems. Morphological operations further enhanced the binary image by modifying object boundaries, reducing noise, and improving region continuity. Since direct visualization was not available on the microcontroller, UART-based data transmission provided an effective method for result verification on a host computer.

Overall, this study highlights the practicality of embedded image processing and shows that meaningful image analysis can be achieved even under strict hardware constraints. The implemented methods form a solid foundation for more advanced embedded vision applications such as object detection, feature extraction, and real-time image analysis.